

Terraform with the chkp/checkpoint Provider

Objectives

1. Configure the chkp/checkpoint Terraform provider against the SMS using the web_api context.
2. Use terraform plan to preview infrastructure changes before applying them.
3. Build a Check Point policy declaratively in HCL (package, access rules, hosts, groups).
4. Understand the publish lifecycle and why checkpoint_management_publish + depends_on is required.
5. Use terraform destroy and the state file for safe, declarative cleanup.

Lab Environment

Resource	Address	Credentials
SMS	10.1.1.100	admin / Cpwins!1
GW	10.1.1.111	admin / Cpwins!1
Ubuntu controller (AI-ubuntu)	10.1.3.33	ubuntu / Cpwins!1

The lab is pre-configured by the instructor. Terraform and the chkp/checkpoint provider (v2.12.0) are pre-installed on the Ubuntu controller; the provider plugin is already cached so terraform init runs offline.

Pre-Flight

Terraform brings the declarative configuration model from infrastructure-as-code to Check Point management. You write your policy state in HCL files, terraform plan previews exactly what will change, and terraform apply commits it. Unlike Ansible's playbook-by-playbook execution, Terraform tracks your environment in a state file — so it knows what to add, what to leave alone, and what to destroy.

1. In MobaXterm, open the AI-ubuntu SSH session — address 10.1.3.33, login ubuntu / Cpwins!1. Same controller as Lab 5; the toolchain is already in place.

2. Verify Terraform is ready. Expect a version line back — terraform 1.15+:

```
terraform version
```

3. Create the lab workspace:

```
mkdir -p ~/lab6 && cd ~/lab6
```

Task 1: Provider + First Host + Plan (15 Minutes)

Task 1 wires up the Terraform provider against the SMS, declares a single host as your first resource, and previews the change without applying it. The plan output is Terraform's killer feature — exactly what is going to happen, before anything happens.

1a — Write provider.tf (paste this whole block):

```
cat > provider.tf <<'EOF'
terraform {
  required_providers {
    checkpoint = {
      source = "CheckPointSW/checkpoint"
      version = "~> 2.12"
    }
  }
}

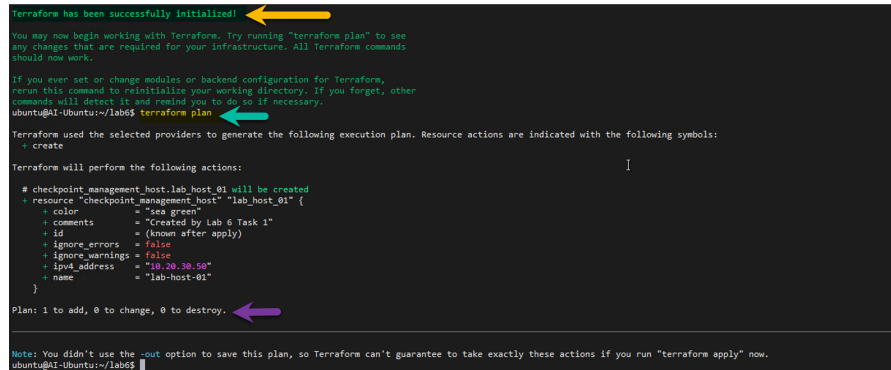
provider "checkpoint" {
  server          = "10.1.1.100"
  username        = "admin"
  password        = "Cpwins!1"
  context         = "web_api"
  session_description = "Lab 6 - Terraform Foundation"
}
EOF
```

1b — Write host.tf:

```
cat > host.tf <<'EOF'
resource "checkpoint_management_host" "lab_host_01" {
  name          = "lab-host-01"
  ipv4_address  = "10.20.30.50"
  color         = "sea green"
  comments      = "Created by Lab 6 Task 1"
}
EOF
```

1c — Initialize Terraform (downloads the provider; fast because it's cached on the snapshot):

```
terraform init
```



Terraform has been successfully initialized! ←

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

ubuntu@AI-Ubuntu:~/lab6\$ terraform plan ←

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

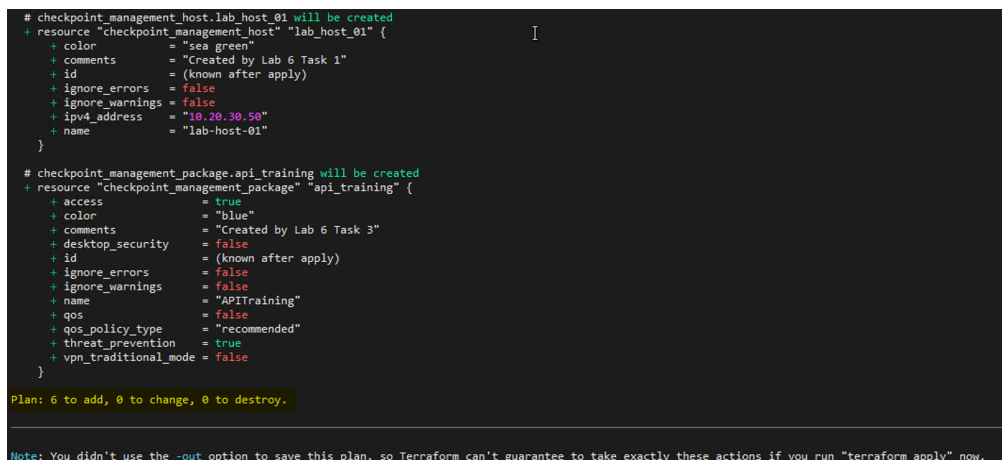
```
# checkpoint_management_host.lab_host_01 will be created
+ resource "checkpoint_management_host" "lab_host_01" {
+   color           = "sea green"
+   comments        = "Created by Lab 6 Task 1"
+   id              = (known after apply)
+   ignore_errors   = false
+   ignore_warnings = false
+   ipv4_address     = "10.20.30.50"
+   name            = "lab-host-01"
}
```

Plan: 1 to add, 0 to change, 0 to destroy. ←

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

1d — Preview the plan. No changes are made yet — this is a dry run:

```
terraform plan
```



```
# checkpoint_management_host.lab_host_01 will be created
+ resource "checkpoint_management_host" "lab_host_01" {
+   color           = "sea green"
+   comments        = "Created by Lab 6 Task 1"
+   id              = (known after apply)
+   ignore_errors   = false
+   ignore_warnings = false
+   ipv4_address     = "10.20.30.50"
+   name            = "lab-host-01"
}

# checkpoint_management_package.api_training will be created
+ resource "checkpoint_management_package" "api_training" {
+   access           = true
+   color            = "blue"
+   comments         = "Created by Lab 6 Task 3"
+   desktop_security = false
+   id              = (known after apply)
+   ignore_errors    = false
+   ignore_warnings  = false
+   name            = "APITraining"
+   qos              = false
+   qos_policy_type  = "recommended"
+   threat_prevention = true
+   vpn_traditional_mode = false
}

Plan: 6 to add, 0 to change, 0 to destroy.
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

Read the output. Terraform reports "Plan: 1 to add, 0 to change, 0 to destroy" and shows the exact attributes of the host it intends to create. SmartConsole is unchanged — nothing happens on the SMS yet. This is the difference from Ansible: with Ansible, --check is opt-in and approximated; with Terraform, every apply is preceded by an implicit plan that you can also run by itself.

Task 2: Add Groups + Re-Plan (10 Minutes)

Same two groups you built in Lab 3 (Bad_IP_List, Good_IP_List), now declarative. Watch the plan grow as you add resources.

```
cat > groups.tf <<'EOF'
resource "checkpoint_management_group" "bad_ip_list" {
  name = "Bad_IP_List"
}

resource "checkpoint_management_group" "good_ip_list" {
  name = "Good_IP_List"
}
EOF

terraform plan
```

Plan now reports "3 to add, 0 to change, 0 to destroy". The host and both groups are queued — still nothing has actually been created on the SMS.

Task 3: Add Policy + Re-Plan (15 Minutes)

Same APITraining package from Lab 3 and Lab 5, now in HCL. Three things to notice: (1) the position dict form { top = "top" } and { below = rule.id } works natively in the Terraform provider — unlike Ansible Lab 5, where the dict was rejected because Ansible wraps it in a list before sending to the API. (2) The dependency chain between resources is automatic — rule_https references rule_ssh.id, which references api_training.name, and Terraform's DAG figures out the apply order without explicit depends_on. (3) The rules declare vpn, install_on, content, and time even though we only care about source/destination/service — this is the Check Point provider quirk: the API populates these fields with defaults server-side; if HCL doesn't match, every plan reports drift. Declaring them explicitly keeps plan clean.

```
cat > policy.tf <<'EOF'
resource "checkpoint_management_package" "api_training" {
  name           = "APITraining"
  access         = true
  threat_prevention = true
  color          = "blue"
  comments       = "Created by Lab 6 Task 3"
}

resource "checkpoint_management_access_rule" "rule_ssh" {
  layer           = "${checkpoint_management_package.api_training.name} Network"
  name            = "Allow SSH/ICMP for admins"
  position        = { top = "top" }
  source          = ["Any"]
  destination     = ["Any"]
  service         = ["ssh", "icmp-proto"]
}
```

```

    action      = "Accept"
    vpn         = ["Any"]
    install_on  = ["Policy Targets"]
    content     = ["Any"]
    time        = ["Any"]

    lifecycle { ignore_changes = [action_settings] }
  }

  resource "checkpoint_management_access_rule" "rule_https" {
    layer      = "${checkpoint_management_package.api_training.name} Network"
    name       = "Allow https for API"
    position   = { below = checkpoint_management_access_rule.rule_ssh.id }
    source     = ["Any"]
    destination = ["Any"]
    service    = ["https"]
    action     = "Accept"
    vpn        = ["Any"]
    install_on = ["Policy Targets"]
    content    = ["Any"]
    time       = ["Any"]

    lifecycle { ignore_changes = [action_settings] }
  }
EOF

terraform plan

```

```

# checkpoint_management_package.api_training will be created
+ resource "checkpoint_management_package" "api_training" {
+   access      = true
+   color       = "blue"
+   comments    = "Created by Lab 6 Task 3"
+   desktop_security = false
+   id          = (known after apply)
+   ignore_errors = false
+   ignore_warnings = false
+   name        = "APITraining"
+   qos         = false
+   qos_policy_type = "recommended"
+   threat_prevention = true
+   vpn_traditional_mode = false
+ }
Plan: 6 to add, 0 to change, 0 to destroy.

```

Plan now reports "6 to add" — one host, two groups, one package, two rules. Still no API calls — Terraform has just been parsing your .tf files into a dependency graph.

Task 4: Publish + Apply (10 Minutes)

This is where Terraform on Check Point differs from Terraform on AWS or Azure. The Management API requires an explicit publish to commit changes — the provider opens a session, creates resources in it, but won't publish unless you tell it to. Without a `checkpoint_management_publish` resource (with `depends_on` on everything else), the

apply succeeds, the session closes, and all your work is discarded. Khalid's repo has a dedicated PDF on this gotcha: Checkpoint_Terraform_publish_behaviour.pdf.

4a — Write publish.tf:

```
cat > publish.tf <<'EOF'
resource "checkpoint_management_publish" "publish" {
  depends_on = [
    checkpoint_management_host.lab_host_01,
    checkpoint_management_group.bad_ip_list,
    checkpoint_management_group.good_ip_list,
    checkpoint_management_package.api_training,
    checkpoint_management_access_rule.rule_ssh,
    checkpoint_management_access_rule.rule_https,
  ]
}
EOF
```

4b — Apply everything. This is the moment changes actually hit the SMS:

```
terraform apply -auto-approve
```

```
# checkpoint_management_package.api_training will be created
+ resource "checkpoint_management_package" "api_training" {
+   access           = true
+   color            = "blue"
+   comments         = "Created by Lab 6 Task 3"
+   desktop_security = false
+   id               = (known after apply)
+   ignore_errors    = false
+   ignore_warnings  = false
+   name             = "APITraining"
+   qos              = false
+   qos_policy_type  = "recommended"
+   threat_prevention = true
+   vpn_traditional_mode = false
+ }

# checkpoint_management_publish.publish will be created
+ resource "checkpoint_management_publish" "publish" {
+   id       = (known after apply)
+   task_id = (known after apply)
+ }

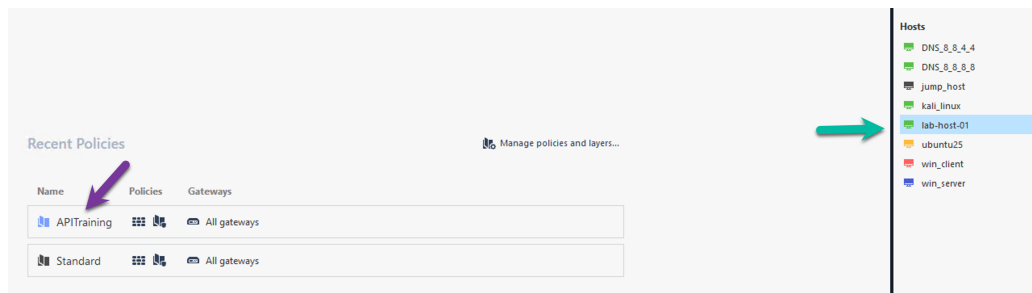
Plan: 7 to add, 0 to change, 0 to destroy.
checkpoint_management_group.good_ip_list: Creating...
checkpoint_management_package.api_training: Creating...
checkpoint_management_group.bad_ip_list: Creating...
checkpoint_management_host.lab_host_01: Creating...
checkpoint_management_host.lab_host_01: Creation complete after 0s [id=d999fdb6-dd56-4d8b-a833-796e27a6ee3e]
checkpoint_management_group.bad_ip_list: Creation complete after 0s [id=ddb1ccb8-b15f-434f-8f57-e66fea6fe624]
checkpoint_management_group.good_ip_list: Creation complete after 0s [id=70b436ac-8d6b-4912-83c8-617e5e2d8bfd]
checkpoint_management_package.api_training: Creation complete after 1s [id=74f1aff2-4e9c-4694-9045-4f0fc4767ecf]
checkpoint_management_access_rule.rule_ssh: Creating...
checkpoint_management_access_rule.rule_ssh: Creation complete after 1s [id=e7e7625a-611b-48d4-bc31-f5baefa1fac7]
checkpoint_management_access_rule.rule_https: Creating...
checkpoint_management_access_rule.rule_https: Creation complete after 0s [id=52577476-063e-453f-a981-8b814d0ed039]
checkpoint_management_publish.publish: Creating...
checkpoint_management_publish.publish: Creation complete after 4s [id=publish-4ks8x6hry0]

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
```

Watch the output. Terraform creates each resource in dependency order, then runs publish last. Total time: ~30-60 seconds.

4c — Verify in SmartConsole:

- Network Objects → Hosts: lab-host-01
- Network Objects → Groups: Bad_IP_List, Good_IP_List
- Security Policies → APITraining: Allow SSH/ICMP (top of layer), Allow https (below it)



Task 5: Idempotency + State + Destroy (10 Minutes)

Run the plan a second time and Terraform will say "No changes" — that's idempotency, but smarter than Ansible's, because it's based on comparing your configuration against the state file (not against the API on every run). Then explore the state file, then destroy everything cleanly.

5a — Re-plan (should be a no-op):

```
terraform plan
```

Expect: "No changes. Your infrastructure matches the configuration."

5b — Inspect the state. Terraform has its own command for this — no jq or file parsing needed. List every tracked resource:

```
terraform state list
```

Expect seven lines back — your host, two groups, package, two rules, and the publish resource. This is Terraform's source of truth: what it created, what it tracks. Ansible has no equivalent.

5c — Destroy everything. Terraform reverses what it created, in reverse dependency order:

```
terraform destroy -auto-approve
```

5d — Verify in SmartConsole that everything is gone. No cleanup playbook needed — destroy reads from state and removes each resource via its Mgmt API delete verb.

Checkpoint (5 Minutes)

For each scenario, write down which tool you would recommend — `mgmt_cli`, Postman, Ansible, or Terraform — and why.

1. A SOC team wants to declare a fleet of host objects, version-control them in Git, and reproduce them in every customer lab they spin up.
2. A CISO wants to see the API live in a meeting; needs visual request/response with response inspection.
3. An SE needs to add three hosts and install policy at 3 AM during a maintenance window; speed matters; the SMS is already in their terminal session.
4. A platform team manages 200+ Check Point objects across 3 environments (dev/staging/prod), wants a single source of truth and the ability to detect drift between configuration and reality.

Expected directions: (1) Ansible — declarative, idempotent, Git-friendly, runs from a controller, but no state file. (2) Postman — visual, exploratory, great for demos. (3) `mgmt_cli` — fastest if you're already on the SMS shell. (4) Terraform — state file enables drift detection, declarative across multiple environments via workspaces or `.tfvars`.

References

6. Provider docs (official):
registry.terraform.io/providers/CheckPointSW/checkpoint/latest/docs
7. Upstream provider source: github.com/CheckPointSW/terraform-provider-checkpoint
8. Khalid's production repo: github.com/alshawwaf/check_point_policy_terraform — module pattern (pre-publish → policy → publish) for production deployments.
9. Publish-behavior write-up: [Checkpoint_Terraform_publish_behaviour.pdf](#) in the same repo — canonical explanation of why publish needs `depends_on`.
10. Deep-dive workshop: [Workshops/Terraform-Workshop/](#) — multi-lab guide covering provider, state, modules, and team workflows.
11. Compare with Ansible: the `check_point.mgmt` collection (Lab 5) covers the same Mgmt API verbs without state tracking — useful contrast for the decision matrix above.